PROJECT REVIEW 1 DOCUMENT
Auto-Vectorizing Loop Optimizer

- **Team Number:** A9
- **Team Members & Register Numbers:**
  - Member 1: Kaviya Shree S – 24BCE2348
  - Member 2: Ayush Garg – 24BCE0275
  - Member 3: Siddhant Jain – 24BCI0190
  - Member 4: Gnana Sasidhar – 24BCE0926
- **Review Date:** 18.10.2026
- **Faculty / Evaluator:** SHALINI L

1. Abstract

Modern numerical, image-processing, and machine-learning workloads spend a large share of their execution time inside simple, repetitive loops that apply the same operation to successive array elements. Production compilers such as LLVM and GCC contain auto-vectorization passes that convert such scalar loops into SIMD instructions automatically, but the internal logic - dependency analysis, legality checking, and code generation - is spread across large, tightly integrated codebases that are difficult for students to study end-to-end. This project implements a smaller, self-contained IR-to-IR auto-vectorizing loop optimizer that demonstrates the same core pipeline on a simplified, custom intermediate representation (IR). The current scope is restricted to single-basic-block counted loops operating on one-dimensional arrays with affine access patterns and element-wise operations, including SAXPY-style computations. The system accepts a loop expressed in the toy IR, identifies it within the supported scope, and uses a simplified GCD/Banerjee-style offset comparison to check for loop-carried dependencies. A legality checker then verifies a defined set of preconditions. Where a loop passes both checks, the vectorizing transformer performs loop strip-mining with a configurable vector width to emit a vector main loop (using VLOAD/VADD/VSTORE-style pseudo-operations) followed by a scalar remainder loop for leftover iterations; loops that fail either check are left in scalar form. The expected outcome is a working, testable prototype whose classification and transformation decisions are verified against a defined suite of known-answer test loops, rather than a claim of measured hardware speedup.

2. Problem Statement and Motivation

2.1 Problem Statement

- Input: the system receives a single counted loop expressed in a simplified toy IR - an induction variable, a trip count, a sequence of statements, and for each array reference, the base array name, an offset expressed relative to the induction variable, and whether the reference is a read or a write.
- Decision: it decides (a) whether the loop's array accesses contain a loop-carried dependency, using an offset comparison between read and write references, and (b) whether the loop satisfies the defined legality preconditions (single basic block, affine indices, supported operation set, no unsupported constructs).
- Transformation: if the loop is legal and dependency-free, the system rewrites it as a vector main loop operating on V elements per iteration plus a scalar remainder loop for the leftover iterations; otherwise the loop is left unchanged in scalar form.
- Correctness condition: for every input that satisfies the stated preconditions, the vectorized output must compute the same array values as the original scalar loop - the transformation must be semantics-preserving and must not introduce a dependency violation.

2.2 Motivation

Loops with independent, per-element array operations are a natural target for SIMD vectorization: the same operation is applied to different data with no control or data dependency between iterations, so a single vector instruction can do the work of several scalar instructions. This pattern is common in numerical computing (vector and matrix arithmetic), image processing (per-pixel filters), and machine-learning kernels (element-wise activation and normalization operations). General-purpose compilers apply vectorization automatically, but only when their internal analysis can prove it is safe to do so; understanding how that analysis decides safety is a core compiler-design skill that this project is built to demonstrate.

It is widely reported in compiler literature that SIMD vectorization can substantially reduce execution time for such workloads on real hardware. This is stated here as general, textbook motivation and not as a result measured by this project - the current scope does not include hardware execution or performance benchmarking before claiming any specific speedup number, since none has been measured yet.

3. Objectives, Scope and Limitations

3.1 Objectives

- Define and implement a toy IR that represents single-basic-block counted loops with one-dimensional, affine array accesses (target: Weeks 2-3).
- Implement a dependency analyzer that classifies a loop's array accesses as dependency-free or loop-carried-dependent using a GCD/Banerjee-style offset comparison, validated against at least 10 hand-written test loops (target: Week 5).
- Implement a legality checker that enforces the stated preconditions (single basic block, affine indices, supported operations, absence of unsupported patterns) before any loop reaches the transformer (target: Week 5).
- Implement a vectorizing transformer that performs loop strip-mining with a configurable vector width V and generates a scalar remainder loop for loops that pass both checks (target: Week 7).
- Validate the end-to-end pipeline on a test suite covering positive, negative, and boundary-case loops (SAXPY, copy, element-wise ops, recurrence, trip-count edge cases), confirming that classification and transformation are correct for each case (target: Weeks 9-10).

3.2 In-Scope Features

- Single-basic-block counted loops
- One-dimensional arrays
- Affine array accesses (index expressed as induction variable times a constant, plus a constant offset)
- Element-wise arithmetic operations (add, subtract, multiply, and similar)
- SAXPY-style operations
- A simplified/toy IR, not full LLVM IR
- A simplified GCD/Banerjee-style dependency test
- Loop strip-mining with a configurable vector width
- Scalar remainder-loop generation

3.3 Out-of-Scope Features

- General pointer alias analysis
- Multi-dimensional loops and arrays
- Reduction operations (e.g., running-sum accumulation)
- Function calls inside loop bodies
- Exception handling
- Data-dependent control flow (if/else) inside the loop body
- Real SIMD code generation or execution on physical hardware
- Loops spanning multiple basic blocks

3.4 Assumptions

- Input loops are already expressed in the toy IR; this project does not build a C or LLVM-IR front-end parser unless explicitly extended later.
- Arrays referenced within a loop do not alias one another unless the IR states otherwise; this is a stated assumption of the current scope, not a proven guarantee.
- Trip counts are treated as known at analysis time for the purpose of remainder-loop generation.

4. Literature / Tool Survey

4.1 Relevant Compiler Concepts

- Intermediate Representation (IR): a structured, lower-level program representation used between the source language and machine code, on which analyses and transformations are performed.
- Basic blocks and loops: a basic block is a straight-line sequence of instructions with a single entry and exit; a counted loop repeats a basic block a known number of times under an induction variable.
- Loop-carried dependencies and RAW/WAR/WAW: a dependency exists when one iteration's memory access must complete before another iteration's access to the same location - read-after-write, write-after-read, and write-after-write are the three classic hazard types that block reordering or parallel execution.
- Affine array accesses: an array index that is a linear function of the loop induction variable (index = c1*i + c0), which is the pattern most dependency tests and vectorizers are built to reason about.
- Dependence testing and GCD/Banerjee-style tests: methods that compare the offsets of array accesses across loop iterations to prove or disprove the existence of a dependency; the GCD test checks a necessary integer-solvability condition, while Banerjee-style tests bound the possible distance between accesses.
- Loop transformation and SIMD/vectorization: rewriting a loop so that several logical iterations are executed by one vector (SIMD) instruction, valid only when no dependency prevents reordering.
- Loop strip-mining: splitting a loop's iteration space into full-width vector chunks and a smaller remainder, which is the standard technique used to handle trip counts that are not exact multiples of the vector width.

4.2 Existing Tools and Approaches

System	What it does	Relation to this project	What we are simplifying
LLVM Loop Vectorizer	Production auto-vectorization pass operating on LLVM IR, with cost-model-driven decisions and target-specific SIMD code generation.	Closest real-world analogue to our pipeline stage order (identify loop, check dependencies/legality, transform).	We use a small toy IR instead of full LLVM IR, a simplified dependency test instead of LLVM's full dependence analysis, and no target-specific code generation or cost model.
GCC Tree-Vectorizer	Auto-vectorization pass operating on GCC's GIMPLE IR, integrated with GCC's broader loop-	Another production reference point for the same overall pipeline shape.	We do not integrate with a real front-end, and our legality checks cover a much smaller

System	What it does	Relation to this project	What we are simplifying
	optimization framework.		precondition set.
Polly (LLVM polyhedral optimizer)	Performs polyhedral-model loop analysis and transformation (including some vectorization-enabling transformations) for affine loop nests in LLVM.	Shows a more mathematically general approach to affine loop analysis than the simplified offset comparison used here.	We do not use the polyhedral model; our project is restricted to single loops, not general loop nests, and uses a simplified offset-based test rather than integer linear programming.

4.3 Identified Gap

Production vectorizers such as LLVM's and GCC's are large systems integrated with full compiler infrastructure, which makes it hard to study dependency analysis, legality checking, and transformation as separable, inspectable steps. This project's gap/contribution is educational rather than a research novelty claim: it aims to build a smaller, inspectable IR-to-IR vectorizer where each of these three stages can be understood, tested, and demonstrated independently on a restricted but realistic loop pattern set.

4.4 Justification of Selected Technologies

- Python: chosen for fast prototyping of the IR data structures and analysis passes.
- A custom toy IR rather than real LLVM IR: keeps the project scope achievable within the semester and keeps every stage inspectable for the viva, at the cost of realism.
- pytest (or an equivalent test runner): used to encode the known-answer test suite described in Section 12.

5. Requirements and Feasibility

5.1 Software Requirements

- Python 3.x
- VS Code (or equivalent editor)
- Git and GitHub for version control
- pytest (or equivalent) for the automated test suite
- LLVM/opt - only if the team actually intends to compare against or eventually target real LLVM IR. .

5.2 Hardware Requirements

No specialized hardware is required for the initial prototype. Vectorization is represented and simulated within the toy IR using pseudo-operations (VLOAD/VADD/VSTORE); no target-specific SIMD instructions are generated or executed on real hardware in the current scope. A standard laptop capable of running Python and Git is sufficient.

5.3 Test Programs / Inputs

- Element-wise operations, e.g. c[i] = a[i] + b[i]
- SAXPY-style operations, e.g. y[i] = a * x[i] + y[i]

- Simple array copy, e.g. b[i] = a[i]
- Dependency-producing recurrence, e.g. a[i] = a[i-1] + 1
- Boundary cases: trip count smaller than vector width, trip count not divisible by vector width, empty loop

5.4 Constraints

- Analysis and transformation are restricted to the toy IR described in Section 3; no real front-end or back-end is in scope.
- Single basic block, one-dimensional arrays, affine access only.
- Twelve-week project timeline and a four-member team with varying prior exposure to compiler internals.
- No dedicated performance-measurement or profiling infrastructure is available for the current scope.

5.5 Feasibility

The scope is feasible for a student project because it deliberately excludes the hardest parts of real vectorizers (general alias analysis, multi-dimensional loops, reductions, and real SIMD code generation) while still requiring the team to implement and integrate the three core analysis/transformation stages that make vectorization possible. Each stage (dependency analysis, legality checking, transformation) is independently testable, which supports incremental development and review-wise demonstration.

5.6 Risks and Mitigation

Risk	Mitigation
Incorrect dependence classification (false positive/negative)	Build the known-answer test suite (Section 12) early and run it after every change to the dependency analyzer.
Loop patterns outside the stated scope appear in test/demo inputs	Enforce the legality checker as a hard gate before any loop reaches the transformer; reject with a clear reason rather than mis-transforming.
Scope creep toward handling full/real LLVM IR	Keep the toy IR frozen once agreed in Weeks 2-3; treat any LLVM IR interoperability as future work (Section 13), not a Review 1-3 deliverable.
Integration problems between modules owned by different members	Agree on the IR data structure and module interfaces (Section 7/8) before implementation starts; Gnana Sasidhar coordinates the integration plan and test harness.
Uneven workload across the team	Team to review Section 9 before Review 1 and adjust supporting-responsibility assignments if the core-design load on one member is too high.

6. Proposed Methodology

Stage 1 - IR Construction

The toy IR stores, for each loop: the induction variable and its trip count; a sequence of statements executed once per iteration; and, for each array reference within those statements, the base array name, the read/write direction, and an offset expressed relative to the induction variable (e.g., a[i], a[i-1],

a[i+1]). This information is exactly what the later stages need and nothing more, keeping the IR intentionally minimal.

Stage 2 - Loop Identification / Scope Check

In the current scope, loops are supplied directly in the toy IR rather than recovered from a control-flow graph (CFG). This stage checks that the supplied loop fits the supported shape: a single basic block, a single counted induction variable, and no unsupported constructs (function calls, exceptions, or data-dependent branches). Recovering natural loops automatically from a general CFG is treated as a future extension, not a Review 1-3 deliverable, unless the team decides otherwise

Stage 3 - Dependency Analysis

This stage collects every array read and write in the loop body and compares their offsets relative to the induction variable, using a simplified GCD/Banerjee-style test to decide whether two accesses to the same array can refer to the same memory location across different iterations.

Vectorizable example: c[i] = a[i] + b[i] - every access uses the same offset (i) within its own iteration; there is no dependency between iterations, so the loop is a candidate for vectorization.

Non-vectorizable example: a[i] = a[i-1] + 1 - the write to a[i] and the read of a[i-1] refer to the same array with different offsets one apart, so a later iteration's read depends on an earlier iteration's write. This loop-carried dependency means the loop cannot be safely vectorized as-is.

The current implementation is intentionally limited to this offset-comparison style of test; it is not a full symbolic or polyhedral dependence solver, and its coverage should not be overstated in the presentation.

Stage 4 - Legality Check

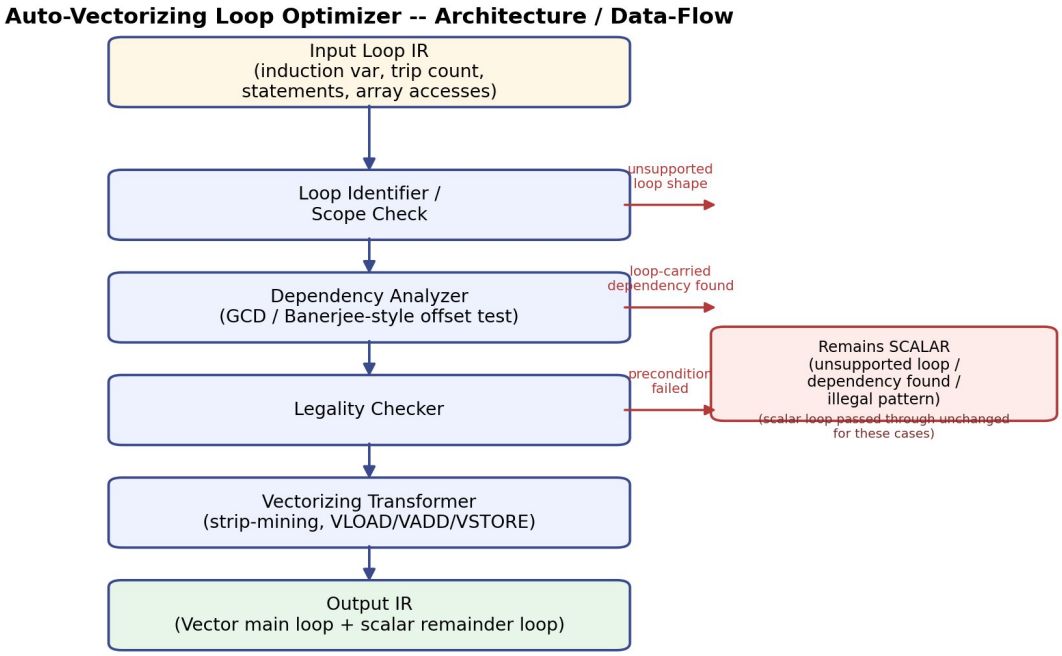
Before any loop reaches the transformer, the legality checker verifies project-specific preconditions: the loop is a single basic block; all array indices are affine in the induction variable; only supported operations appear in the loop body; no unsupported pattern (function call, exception, data-dependent branch) is present; and the stated no-aliasing assumption holds for the arrays involved. A loop failing any precondition is left in scalar form.

Stage 5 - Vectorizing Transformation

For a loop that passes both checks, the transformer chooses a vector width V and performs loop strip-mining: the trip count is split into full-width chunks of V iterations, each replaced by vector pseudo-operations (VLOAD, VADD/VSUB/VMUL as appropriate, VSTORE), plus a scalar remainder loop that processes any leftover iterations when the trip count is not an exact multiple of V.

7. Architecture and Data Flow

The diagram below shows the pipeline stages in order, and the three points at which a loop can be rejected and left in scalar form rather than proceeding to vectorization.



Information passed between stages: the Loop Identifier passes a confirmed in-scope loop description (or a rejection reason) to the Dependency Analyzer. The Dependency Analyzer passes the same loop description plus a dependency verdict (and, if dependent, which accesses conflict) to the Legality Checker. The Legality Checker passes the loop description plus a legality verdict (and, if illegal, which precondition failed) to the Vectorizing Transformer. The Vectorizing Transformer emits the Output IR: either the original scalar loop unchanged (any rejection path) or a vector main loop plus scalar remainder loop (legal, dependency-free path).

8. Module Description

Module	Input	Processing	Output	Owner
IR / Loop Model	Raw loop description (source or hand-written test)	Structures the loop into the toy IR: induction variable, trip count, statements, array accesses	Toy-IR loop object	Siddhant Jain (core design), with integration support from Kaviya Shree S
Loop Identifier / Scope Check	Toy-IR loop object	Checks the loop is a single basic block with a supported induction-variable/trip-count	Confirmed in-scope loop, or rejection reason	Siddhant Jain

Module	Input	Processing	Output	Owner
		form		
Dependency Analyzer	In-scope loop object	GCD/Banerjee-style offset comparison across array accesses	Dependency verdict (dependency-free / loop-carried dependency found)	Siddhant Jain
Legality Checker	Loop object + dependency verdict	Verifies affine indices, supported operations, absence of unsupported patterns, aliasing assumption	Legality verdict (legal / illegal + reason)	Siddhant Jain
Vectorizing Transformer	Loop object + legal & dependency-free verdicts	Loop strip-mining with vector width V; emits vector main loop + scalar remainder loop	Output IR (vectorized or unchanged scalar loop)	Siddhant Jain
Test Harness	Test-suite loop inputs + expected classifications/outputs	Runs the pipeline on each test case and compares actual vs. expected result	Pass/fail report per test case	Gnana Sasidhar
Repository / Documentation / Build Support	Team's code, docs, and meeting notes	Maintains repo structure, README, contribution log, and this document	Organized repository and up-to-date documentation	Gnana Sasidhar, with support from Kaviya Shree S

9. Team Responsibility Matrix

Member	Primary Responsibility	Supporting Responsibility	Review 1 Evidence	Review 2 Target	Review 3 Target
Kaviya Shree S	Problem definition, motivation, objectives, scope and limitations, project-level requirements	Integration and testing support	Problem-definition notes, objective matrix, scope definition, requirement-understanding notes	Support integration testing across modules; help validate requirement coverage	Support fina documentati and consiste check agains implementat
Ayush Garg	Compiler background, literature/tool survey, existing-approaches comparison,	Test-case authoring support	Literature survey, existing-system comparison, references, requirements/environment evidence, initial test-case ideas	Expand test-case set; refine feasibility notes based on progress	Support fina references ar background-section polis

Member	Primary Responsibility	Supporting Responsibility	Review 1 Evidence	Review 2 Target	Review 3 Target
	requirements, feasibility				
Siddhant Jain	Core system design: toy IR/loop model, dependency analyzer, legality checker, vectorizing transformer, architecture and algorithm specification	Explaining module dependencies during integration	Architecture diagram, module specification, algorithm/pseudocode	Working dependency analyzer + legality checker with passing tests	Working vectorizing transformer integrated en to-end
Gnana Sasidhar	Repository structure, test harness, test planning, timeline, integration planning, documentation coordination	Initial implementation/prototype support	Repository/folder structure, test-plan draft, timeline/Gantt chart, integration plan	Test harness running against Weeks 3-7 modules; updated timeline	Full test suit results and integration sign-off for final demo

10. Timeline and Review-Wise Deliverables

Weeks	Planned Activities
1-2	Problem study; IR design; literature survey; repository/environment setup; Review 1 preparation
3-5	Loop model, dependency analyzer, legality checker
6-7	Vectorizing transformer, strip-mining, remainder loop - Review 2 target
8-9	Strided accesses, SAXPY, boundary cases
10	Correctness verification
11	Performance/speedup simulation; documentation
12	Final integration; final demo; Review 3

12. Testing Strategy

Test ID	Category	Input Pattern	Expected Result	Reason
T-01	Positive	c[i] = a[i] + b[i]	VECTORIZABLE	Independent per-element read/write, no shared offsets

Test ID	Category	Input Pattern	Expected Result	Reason
T-02	Positive	c[i] = a[i] - b[i]	VECTORIZABLE	Same pattern with subtraction
T-03	Positive	c[i] = a[i] * b[i]	VECTORIZABLE	Same pattern with multiplication
T-04	Positive	b[i] = a[i] (copy)	VECTORIZABLE	Single read, single write, same offset
T-05	Positive	y[i] = alpha * x[i] + y[i] (SAXPY)	VECTORIZABLE	Read/write of y[i] use the same offset in the same iteration; no cross-iteration dependency
T-06	Negative	a[i] = a[i-1] + 1	NOT VECTORIZABLE	Loop-carried RAW dependency (offset -1)
T-07	Negative	a[i+1] = a[i] * 2	NOT VECTORIZABLE	Cross-offset access creates a dependency across iterations
T-08	Negative	Loop body containing a function call	REJECTED (out of scope)	Function calls are explicitly out-of-scope for the legality checker
T-09	Boundary	Trip count smaller than vector width V	Scalar remainder loop only, no vector main loop executes	Not enough iterations to fill one vector chunk
T-10	Boundary	Trip count not divisible by V	Vector main loop + non-empty scalar remainder loop	Standard strip-mining split
T-11	Boundary	Empty loop (trip count = 0)	No-op output; pipeline must not fail	Degenerate case that a naive implementation can crash on

13. Expected Results and Extension Potential

13.1 Expected Final Outcome

A working, scoped IR-to-IR vectorizer that: (1) accepts supported loop IR; (2) identifies relevant array accesses; (3) detects the supported dependency patterns; (4) performs legality checking; (5) vectorizes

eligible loops; (6) generates a scalar remainder loop; and (7) validates its own results using the known-answer test suite in Section 12.

13.2 Future Extensions (explicitly not part of the current deliverable)

- Reduction-operation vectorization
- Multi-dimensional array support
- Accepting real LLVM IR as input
- A more complete pointer/array alias analysis
- Real SIMD lowering and hardware execution
- Hardware performance evaluation

14. References

- Aho, A. V., Lam, M. S., Sethi, R., and Ullman, J. D. Compilers: Principles, Techniques, and Tools.
- Allen, R., and Kennedy, K. Optimizing Compilers for Modern Architectures.
- LLVM Loop Vectorizer documentation (LLVM Project).
- GCC Auto-Vectorization documentation (GNU Project).
- Banerjee, U. Dependence Analysis for Supercomputing.

Faculty-Requirement Compliance Audit

This table checks the document above against each of the seven graded Review 1 components. It is meant to be read before submission, not presented as a slide.

Faculty Requirement	Where Covered	Evidence Available?	Action Needed
1. Problem Definition, Motivation and Scope (2.0)	Sections 2 and 3	Content drafted; objectives are realistic but team must confirm register numbers, team number and review date	Fill cover-page placeholders; team to review objective wording before presenting
2. Background Study and Technical Understanding (1.5)	Section 4	Concepts and tool comparison drafted from general compiler-design knowledge	Team (esp. Ayush) should verify tool-comparison details independently and be ready to defend them in viva
3. Proposed Methodology and System Design (2.0)	Sections 6, 7, 8	Methodology and architecture diagram drafted from the stated pipeline; no implementation evidence attached	Siddhant must be able to explain every module and the diagram without reading slides
4. Requirements and Feasibility (1.0)	Section 5	Software/hardware requirements listed; LLVM/opt usage not confirmed	**[TO BE COMPLETED BY TEAM]**: confirm whether LLVM/opt integration is actually planned
5. Work Allocation and Timeline (1.5)	Sections 9, 10	Responsibility matrix and 12-week plan drafted per team's stated allocation	Team must confirm/adjust workload balance (Section 9 note) and set real dates
6. Initial Progress / Proof of Start (1.0)	Section 11	NO EVIDENCE PROVIDED - no repository, code, or screenshots were shared with this draft	**[TO BE COMPLETED BY TEAM]**: this is the weakest section right now and needs real evidence before submission
7. Presentation, Individual Contribution and Viva (1.0)	Section 9; PPT structure; viva concept lists	Framework provided; actual readiness depends on each member's preparation	Each member should rehearse their assigned slides and viva concepts (see final checklist)